



UNIVERSIDAD DE LAS FUERZAS ARMADAS

ESPE

Departamento de Ciencias de la Computación.

INTEGRANTES: Martin Palacios
Matias Medina
Leopoldo Bolanos

FECHA: 18 de junio del 2026.

GRUPO: Grupo 7

ASIGNATURA: Metodología de desarrollo de Software.

Modelado UML, Diagrama de casos de uso, Diagrama Puente

DOCENTE: Mgtr. Jenny Ruiz.

NRC: 30693

1.	REQUISITOS FUNCIONALES POR SPRINT	4
2.	SPRINT 1	4
2.1.	DIAGRAMA DE CASOS DE USO	4
2.2.	DIAGRAMA PUENTE.....	6
3.	SPRINT 2	ERROR! BOOKMARK NOT DEFINED.
3.1.	DIAGRAMA DE CASOS DE USO	8
3.2.	DIAGRAMA PUENTE.....	11
4.	SPRINT 3	19
4.1.	DIAGRAMA DE CASOS DE USO	19
4.2.	DIAGRAMA PUENTE.....	21

1. Listado de Requisitos del Sistema de Gestión de Caja para TAGS y Productos

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
REQ001	El sistema deberá permitir el acceso del administrador mediante usuario y contraseña.	Administrador/ Cajero	Iniciar sesión
REQ002	El sistema deberá permitir la gestión de productos de tienda para agregarlos al cierre de caja.	Administrador/ Cajero	Gestionar productos
REQ003	El sistema deberá permitir el registro de clientes con validación de datos.	Administrador/ Cajero	Registrar cliente
REQ004	El sistema deberá permitir la gestión de servicios TAG.	Administrador/ Cajero	Gestionar servicios TAG
REQ005	El sistema deberá permitir la gestión de servicios variables con concepto y valor.	Administrador/ Cajero	Gestionar servicios variables
REQ006	El sistema deberá permitir consultar y guardar el cierre final de caja en una base de datos no relacional.	Administrador/ Cajero	Cierre de caja
REQ007	El sistema debe permitir ingresar un valor numérico llamado caja chica en el modulo de cierre de caja	Administrador/ Cajero	Caja Chica
REQ008	El sistema deberá permitir buscar por fecha los cierres de caja existentes.	Administrador/ Cajero	Busqueda por Fecha
REQ009	El sistema deberá permitir filtrar por método de pago las ventas que se realice durante una jornada para poder diferenciar el dinero que se tiene en efectivo	Administrador/ Cajero	Metodo de Pago

1. Requisitos Funcionales por Sprint

Sprint	Requisito funcional
Sprint 1	REQ001 - Acceso al sistema
Sprint 1	REQ002 - Gestión de productos
Sprint 1	REQ003 - Registro de clientes
Sprint 2	REQ004 - Gestión de servicios TAG
Sprint 2	REQ005 - Gestión de servicios variables
Sprint 2	REQ006 - Cierre de caja
Sprint 3	REQ007 – Gestion Caja Chica
Sprint 3	REQ008 - Búsqueda de Cierres de Caja por fecha
Sprint 3	REQ009 – Gestion de Metodos de Pago

2. Sprint 1

2.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Cajero

rectangle "Sistema de Gestion de Caja - Sprint 1" {
    usecase "RE0001: Iniciar Sesión" as CU1
    usecase "REQ002: Seleccionar Productos y Carrito" as CU2
    usecase "REQ003 Gestionar Cartera de Clientes" as CU3
}

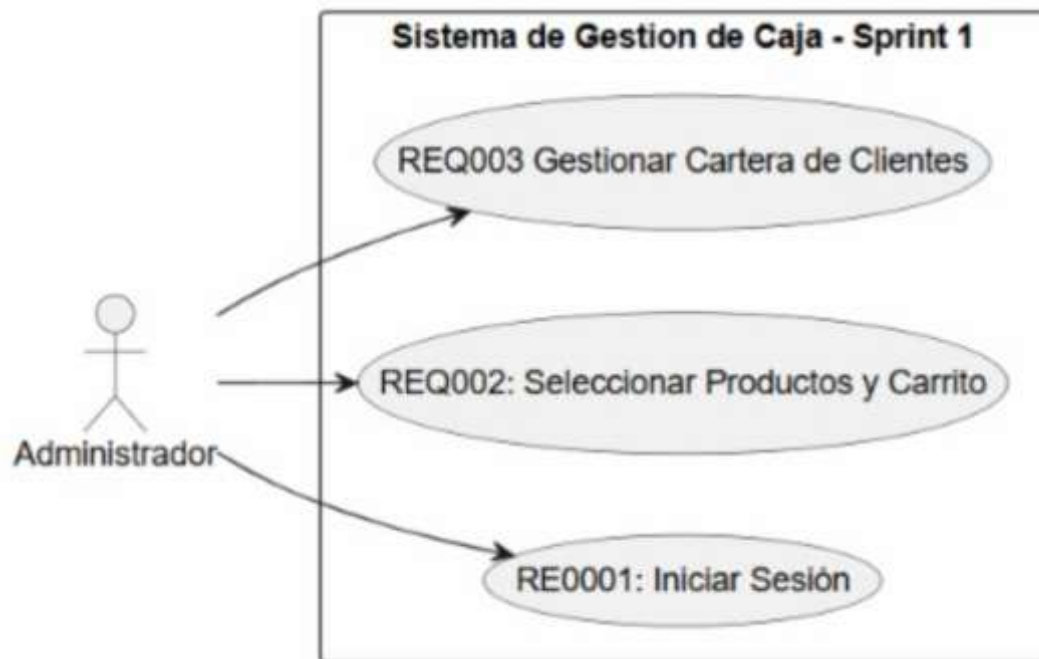
Cajero --> CU1
```

Cajero --> CU2

Cajero --> CU3

@enduml

Diagrama de casos de uso en PlantUML



2.2.Diagrama Puente

Código Puente PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administradora" as Cajero

rectangle "Diagrama Puente - Sprint 1 " {

    ' CU-01
    boundary "LoginView" as B_Login
    control "LoginController" as C_Login
    entity "LoginModel / UserRepository" as E_User

    ' CU-02
    boundary "SalesPanel" as B_Sales
    control "SalesController" as C_Sales
    entity "SalesJournalModel / CurrentSaleModel" as E_Sales

    ' CU-03
    boundary "ClientPanel" as B_Client
    control "ClientController" as C_Client
    entity "ClientModel / ClientRepository" as E_Client

}

' Flujo CU-01: Iniciar Sesión
Cajero --> B_Login : 1. Ingresa credenciales
B_Login --> C_Login : 2. Solcita autenticación
C_Login --> E_User : 3. Consulta usuario

' Flujo CU-02: Seleccionar Productos
Cajero --> B_Sales : 1. Selecciona / Ajusta productos
B_Sales --> C_Sales : 2. Notifica acción
```

C_Sales --> E_Sales : 3. Actualiza carrito/subtotal

' Flujo CU-03: Cartera de Clientes

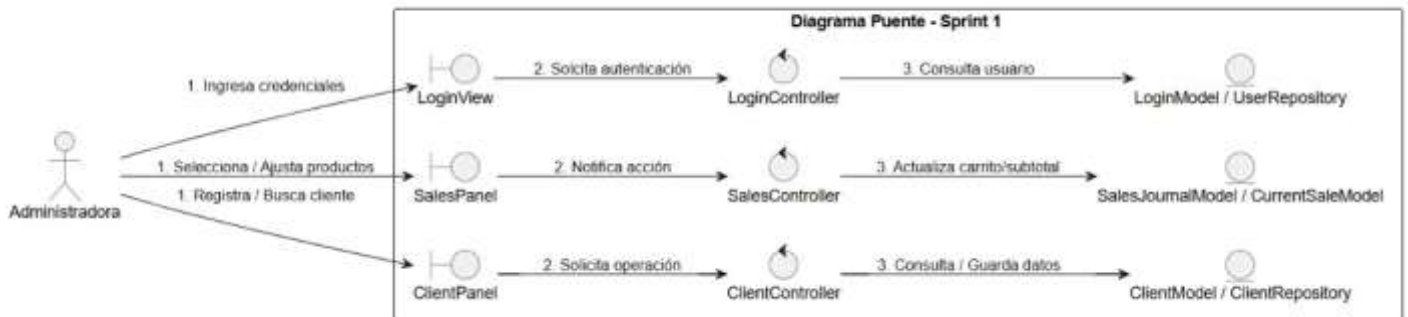
Cajero --> B_Client : 1. Registra / Busca cliente

B_Client --> C_Client : 2. Solicita operación

C_Client --> E_Client : 3. Consulta / Guarda datos

@enduml

Diagrama de Puente



2.3.Diagrama de clases

Código en PlantUML

```
@startuml
skinparam classAttributeIconSize 0

package "Módulo Autenticación (REQ001)" {
    class LoginModel {
        - UserRepository userRepository
        + LoginModel()
        + authenticate(username: String, password: String): boolean
    }

    class UserRepository {
        + UserRepository()
        + findUser(username: String): Document
        + validatePassword(rawPassword: String, storedHash: String): boolean
    }
}

package "Módulo Ventas y Carrito (REQ002)" {
    class SalesJournalModel {
        - Map<String, Double> productCatalog
        - Map<String, Integer> quantities
        + SalesJournalModel()
        + addProduct(productName: String): void
        + removeProduct(productName: String): void
        + hasProducts(): boolean
        + getSubtotal(): double
        + getFormattedReport(): String
        + clearProducts(): void
    }

    class CurrentSaleModel {
```



```

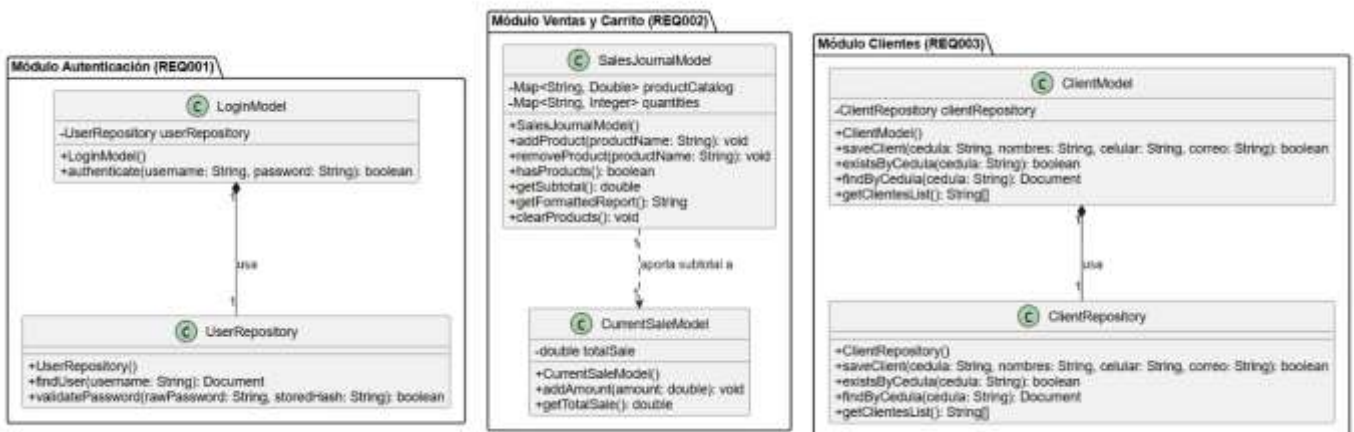
        - double totalSale
        + CurrentSaleModel()
        + addAmount(amount: double): void
        + getTotalSale(): double
    }
}

package "Módulo Clientes (REQ003)" {
    class ClientModel {
        - ClientRepository clientRepository
        + ClientModel()
        + saveClient(cedula: String, nombres: String, celular: String, correo:
String): boolean
        + existsByCedula(cedula: String): boolean
        + findByCedula(cedula: String): Document
        + getCientesList(): String[]
    }
    class ClientRepository {
        + ClientRepository()
        + saveClient(cedula: String, nombres: String, celular: String, correo:
String): boolean
        + existsByCedula(cedula: String): boolean
        + findByCedula(cedula: String): Document
        + getCientesList(): String[]
    }
}

' Relaciones de Composición / Asociación dentro de cada subsistema
LoginModel "1" *-- "1" UserRepository : usa
ClientModel "1" *-- "1" ClientRepository : usa
SalesJournalModel "1" ..> "1" CurrentSaleModel : aporta subtotal a
@enduml

```

Diagrama de clases



3. Sprint 2

3.1. Diagrama Casos de Uso

Codigo en PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Cajero / Usuario" as Usuario

rectangle "Sistema de Caja y Ventas - Sprint 2" {

    ' REQ004: Gestión de Servicios TAG
    usecase "Registrar Servicio TAG" as UC_TAG
    usecase "Validar Cantidad y Precio TAG" as UC_ValTAG
    usecase "Agregar Servicio TAG a Venta Actual" as UC_AddTAG

    ' REQ005: Gestión de Servicios Variables
    usecase "Registrar Servicio Variable" as UC_Var
    usecase "Validar Concepto y Monto Variable" as UC_ValVar
    usecase "Agregar Servicio Variable a Venta Actual" as UC_AddVar

    ' REQ006: Gestión de Cierre de Caja
    usecase "Consultar Reporte Final del Día" as UC_VerReporte
    usecase "Guardar Cierre de Jornada" as UC_CierreJornada
    usecase "Validar Existencia de Movimientos" as UC_ValMov

    ' Relaciones REQ004
    Usuario --> UC_TAG
    UC_TAG ..> UC_ValTAG : <<include>>
    UC_TAG ..> UC_AddTAG : <<include>>
}
```

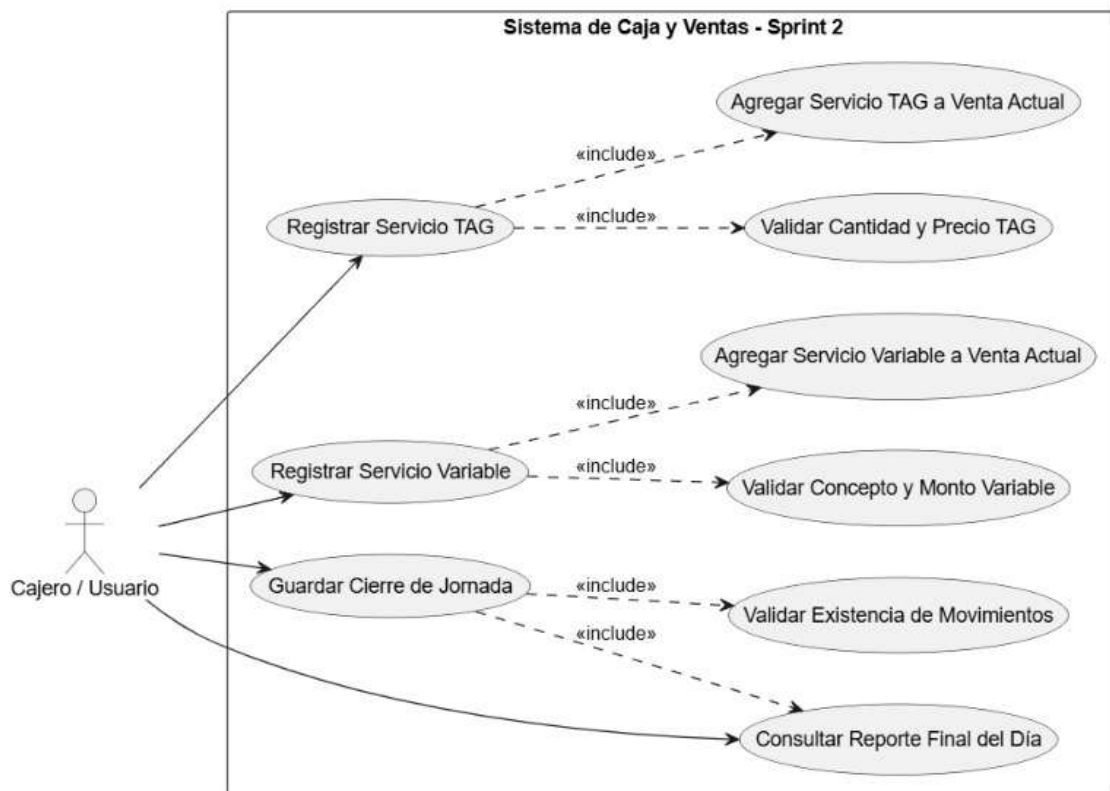
```

' Relaciones REQ005
Usuario --> UC_Var
UC_Var ..> UC_ValVar : <<include>>
UC_Var ..> UC_AddVar : <<include>>

' Relaciones REQ006
Usuario --> UC_VerReporte
Usuario --> UC_CierreJornada
UC_CierreJornada ..> UC_ValMov : <<include>>
UC_CierreJornada ..> UC_VerReporte : <<include>>
}
@endl

```

Diagrama de casos de uso en PlantUML



4. Sprint 2

4.1. Diagrama Puente

Codigo Puente PlantUML

```
@startuml
top to bottom direction
actor "Administradora" as Usuario
package "Vistas" {
    boundary "TagServicePanel" as TagView
    boundary "VariableServicePanel" as VarView
    boundary "CashFlowPanel" as CashView
}
package "Controladores" {
    control "TagServiceController" as TagCtrl
    control "VariableServiceController" as VarCtrl
    control "CashFlowController" as CashCtrl
}
package "Modelos y Datos" {
    entity "CurrentSaleModel" as SaleModel
    entity "CashFlowModel" as CashModel
    entity "CashFlowRepository" as Repository
}
' Flujo REQ004: TAG
Usuario --> TagView : Registrar servicio TAG
TagView --> TagCtrl
TagCtrl --> SaleModel : addTagService()
' Flujo REQ005: Variable
Usuario --> VarView : Registrar servicio variable
VarView --> VarCtrl
```

VarCtrl --> SaleModel : addVariableService()

' Flujo REQ006: Cierre

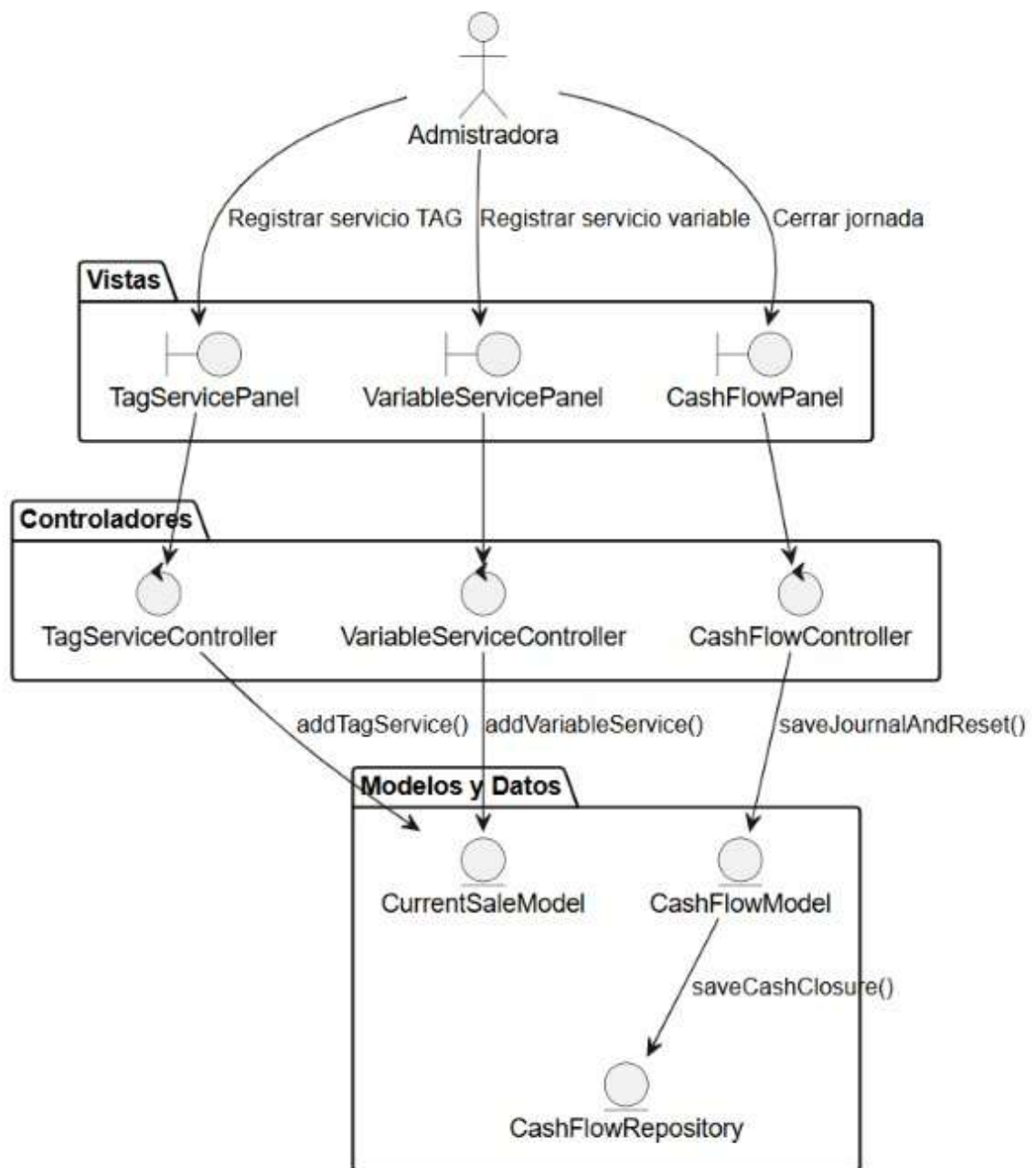
Usuario --> CashView : Cerrar jornada

CashView --> CashCtrl

CashCtrl --> CashModel : saveJournalAndReset()

CashModel --> Repository : saveCashClosure()

@endl

Diagrama de puente en PlantUML

5. Sprint 2

5.1. Diagrama de clases

Codigo Diagrama de clases PlantUML4

```
@startuml
skinparam classAttributeIconSize 0

package "Vistas" {
    class TagServicePanel
    class VariableServicePanel
    class CashFlowPanel
}

package "Controladores" {
    class TagServiceController {
        + addTagService()
    }
    class VariableServiceController {
        + addVariableService()
    }
    class CashFlowController {
        + guardarJornada()
    }
}

package "Modelos" {
    class CurrentSaleModel {
        + addTagService()
        + addVariableService()
    }
}
```



```

class CashFlowModel {
    + getFinalReport()
    + saveJournalAndReset()
}
}

package "Persistencia" {
    class CashFlowRepository {
        + saveCashClosure()
    }
}

' Relaciones principales
TagServiceController --> TagServicePanel
TagServiceController --> CurrentSaleModel

VariableServiceController --> VariableServicePanel
VariableServiceController --> CurrentSaleModel

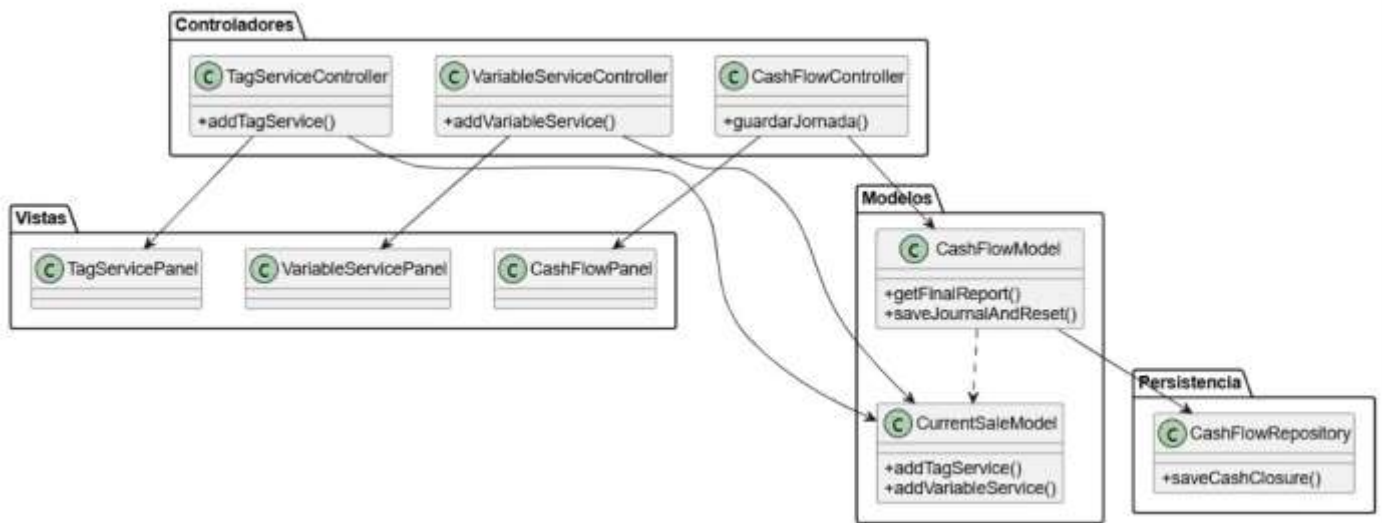
CashFlowController --> CashFlowPanel
CashFlowController --> CashFlowModel

CashFlowModel --> CashFlowRepository
CashFlowModel ..> CurrentSaleModel

@endl

```

Diagrama de clases en PlantUML



6. Sprint 3

6.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administradora" as Usuario

rectangle "Sistema POS - Sprint 3 (REQ007, REQ008, REQ009)" {

    ' REQ007
    usecase "Registrar Caja Chica Actual" as UC_CajaActual
    usecase "Registrar Caja Chica para Mañana" as UC_CajaManana
    usecase "Validar Monto Numérico" as UC_ValMonto

    ' REQ008
    usecase "Buscar Cierres por Fecha" as UC_BuscarFecha
    usecase "Consultar Todos los Cierres" as UC_MostrarTodos
    usecase "Validar Formato de Fecha" as UC_ValFecha

    ' REQ009
    usecase "Procesar Pago de Venta" as UC_ProcesarPago
    usecase "Limpiar Venta Actual" as UC_LimpiarVenta
    usecase "Validar Desglose de Pago Mixto" as UC_ValPagoMixto

    ' Relaciones REQ007
    Usuario --> UC_CajaActual
    Usuario --> UC_CajaManana
}
```

```

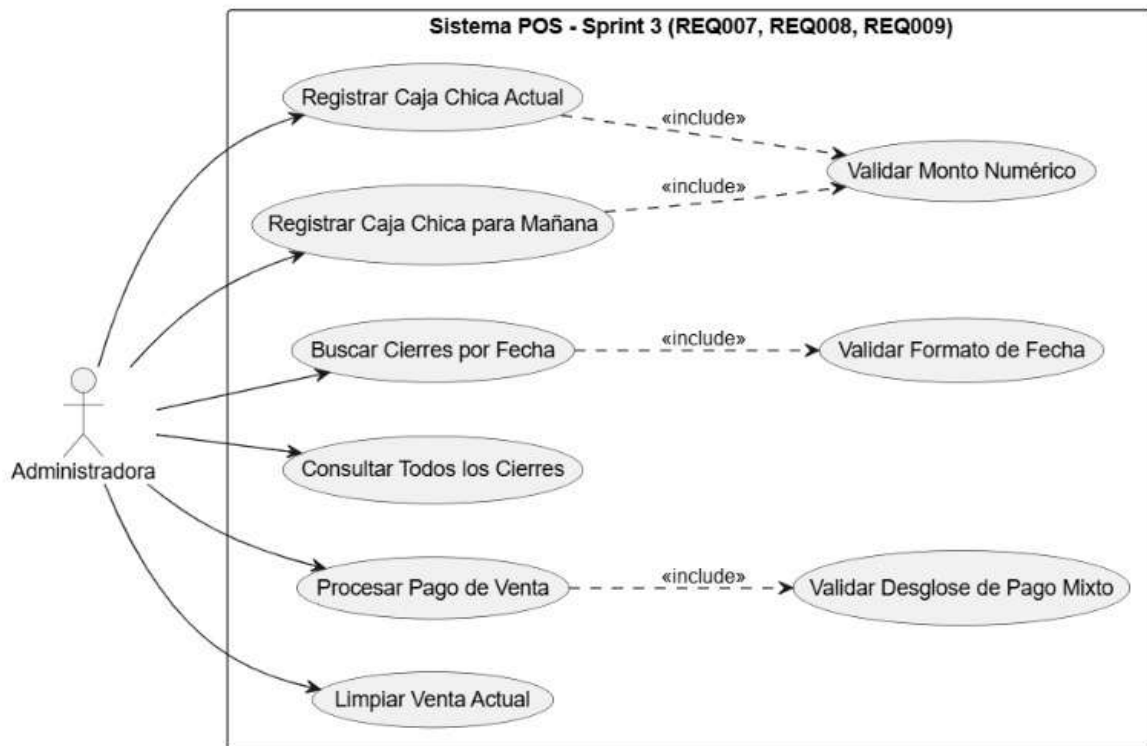
UC_CajaActual ..> UC_ValMonto : <<include>>
UC_CajaManana ..> UC_ValMonto : <<include>>

' Relaciones REQ008
Usuario --> UC_BuscarFecha
Usuario --> UC_MostrarTodos
UC_BuscarFecha ..> UC_ValFecha : <<include>>

' Relaciones REQ009
Usuario --> UC_ProcesarPago
Usuario --> UC_LimpiarVenta
UC_ProcesarPago ..> UC_ValPagoMixto : <<include>>
}
@endl

```

Diagrama de casos de uso en PlantUML



6.2.Diagrama Puente

Código Puente PlantUML

```
@startuml
top to bottom direction

actor "Administradora" as Usuario

package "Vistas" {
    boundary "CashFlowPanel" as FlowView
    boundary "CashHistoryPanel" as HistView
    boundary "PaymentPanel" as PayView
}

package "Controladores" {
    control "CashFlowController" as FlowCtrl
    control "CashHistoryController" as HistCtrl
    control "PaymentController" as PayCtrl
}

package "Modelos y Datos" {
    entity "CashFlowModel" as FlowModel
    entity "CashHistoryModel" as HistModel
    entity "PaymentDetail" as PayDetail
    entity "CashFlowRepository" as Repository
}

' Interacciones de usuario
Usuario --> FlowView : Gestión caja chica
Usuario --> HistView : Consulta historial
Usuario --> PayView : Procesa pago
```

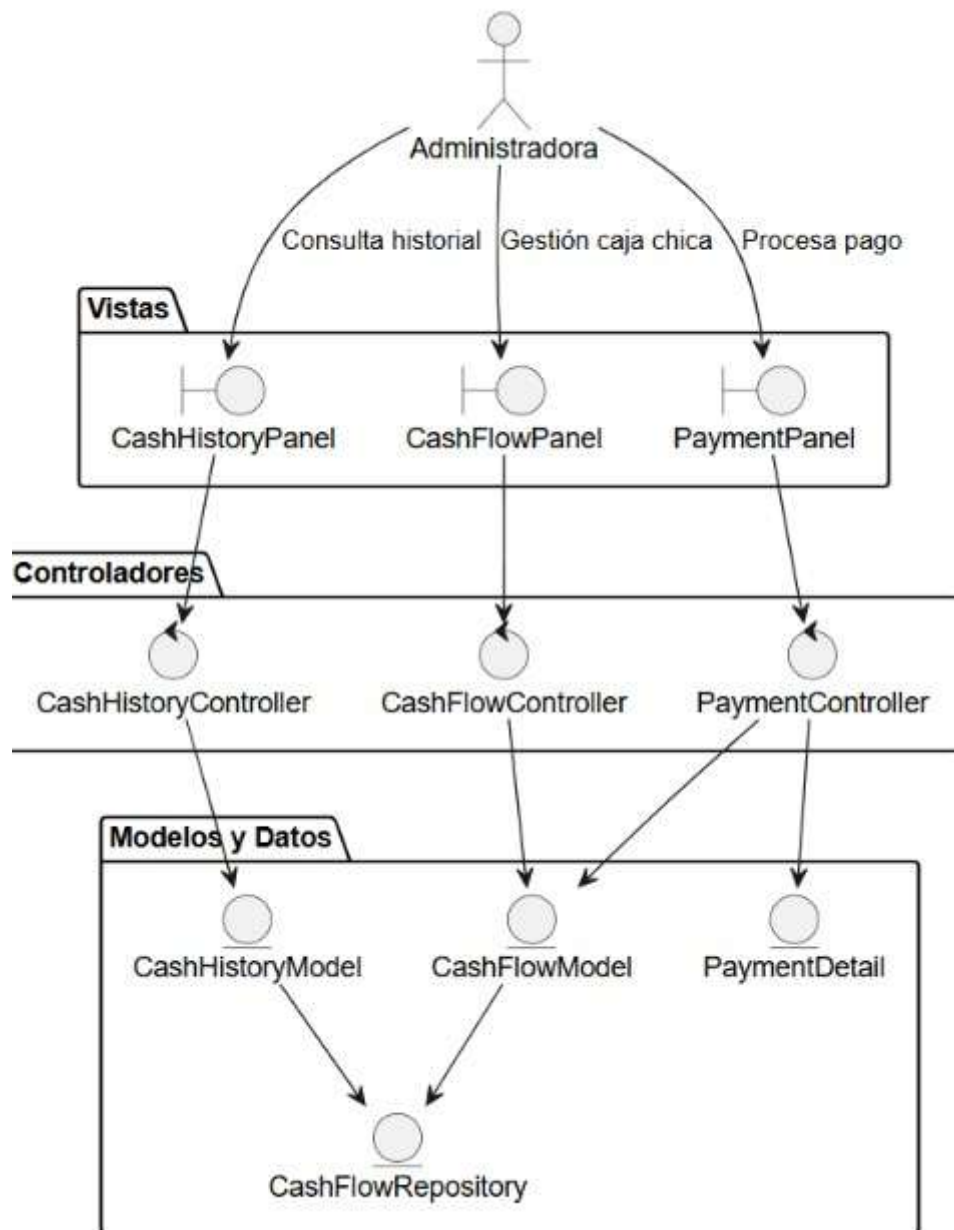
```
' Flujos de control y datos  
FlowView --> FlowCtrl  
FlowCtrl --> FlowModel  
FlowModel --> Repository
```

```
HistView --> HistCtrl  
HistCtrl --> HistModel  
HistModel --> Repository
```

```
PayView --> PayCtrl  
PayCtrl --> PayDetail  
PayCtrl --> FlowModel
```

```
@endl
```

Diagrama puente en PlantUML



6.3.Diagrama de clases

Código diagrama de clases PlantUML

```
@startuml
skinparam classAttributeIconSize 0

package "Vistas" {
    class CashFlowPanel
    class CashHistoryPanel
    class PaymentPanel
}

package "Controladores" {
    class CashFlowController
    class CashHistoryController
    class PaymentController
}

package "Modelos" {
    class CashFlowModel
    class CashHistoryModel
    class CurrentSaleModel
    class PaymentDetail
    enum PaymentMethod
}

package "Persistencia" {
    class CashFlowRepository
}

' Relaciones principales
```


CashFlowController --> CashFlowPanel

CashFlowController --> CashFlowModel

CashHistoryController --> CashHistoryPanel

CashHistoryController --> CashHistoryModel

PaymentController --> PaymentPanel

PaymentController --> CurrentSaleModel

PaymentController --> CashFlowModel

PaymentController ..> PaymentDetail

' Integración con datos y enums

CashFlowModel --> CashFlowRepository

CashHistoryModel --> CashFlowRepository

PaymentDetail --> PaymentMethod

@endl

Diagrama clases en PlantUML

